

Welcome back

How was lab? Was anything confusing?

Some announcements

At the *end* of the class, we will give everyone a flash drive with all of the course material and a way to install Jupyter

Review

Last lecture, we covered:

- Strings
- Booleans
- Types
- Equalities, inequalities
- Arithmetic for non-numbers

```
(-4) ** 0.5
(1.2246467991473532e-16+2j)
type("hello")
str
type(True)
bool
type(10)
int
type(10.0)
float
0 == 0.0
True
float(0) == 0.0
True
```

```
1 == True
True
int(True)
1
0 == "0"
False
True == "True"
False
"hello" == "hello"
True
"hello" == "Hello"
False
"h" == "H"
False
"hello" == "hello "
False
"0.999999" == "0.9999999"
False
"hello" >= "hello"
True
"hello" > "hello"
False
"a" > "b"
False
"a" < "b"
True
"a" < "z"
True
```

```
"a" > "A"
```

```
True
```

```
"z" > "A"
```

```
True
```

```
"ab" > "aa"
```

```
True
```

```
"a" > "aa"
```

```
False
```

```
"a" < "aa"
```

```
True
```

```
"aa" > "a "
```

```
True
```

```
"aA" > "a "
```

```
True
```

```
"a" > 1.0
```

```
-----  
-----
```

```
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[58], line 1
```

```
----> 1 "a" > 1.0
```

```
TypeError: '>' not supported between instances of 'str' and 'float'
```

```
"a" > 0
```

```
-----  
-----
```

```
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[28], line 1
```

```
----> 1 "a" > 0
```

```
TypeError: '>' not supported between instances of 'str' and 'int'
```

```
"hello"
```

```
'hello'
```

```
True
```

```
True
```

```
type(1)
```

```
int
```

```
type(1.0)
```

```
float
```

```
type("hello")
```

```
str
```

```
type(True)
```

```
bool
```

```
1 == 1
```

```
True
```

```
1 != 1
```

```
False
```

```
1 > 1
```

```
False
```

```
1 >= 1
```

```
True
```

```
and
```

```
Cell In[69], line 1
```

```
    and  
    ^
```

```
SyntaxError: invalid syntax
```

```
or
```

```
Cell In[70], line 1
```

```
    or  
    ^
```

```
SyntaxError: invalid syntax
```

```
True and True
```

```
True
True and False
False
False and True
False
False and False
False
True or True
True
True or False
True
False or True
True
False or False
False
"hello" + "addis coder"
'helloaddis coder'
"hello " * 5
'hello hello hello hello hello '
```

Reminder: if you're confused, you can test in Jupyter!

```
# Things that were confusing from earlier today
"ba" > "aa"
True
"a" > "aa"
False
"a" > ""
```

True

Variables

We briefly covered variables yesterday, but we will go through them in depth today.

There are big differences between *mathematical variables* and *programming variables*.

In math, let's say we have $x=1$, $y=1+x$. This fully determines the values of x and y .

Programming variables ARE NOT math variables.

Programming variables hold Python "things".

```
x = 1
x
1

x = 3
y = x
y
3

x = 2
y = 3
x + y
5

x = 2
y = 3
z = x + y
z
5
```

Variables can be reassigned

Because variables in Python are holders of things, they can be reassigned

```
x = 3
x = 4
x
4
```

```
x = 2
x = x + 1
x
3
```

Variables are remembered across cells

```
x
3
z
5
y
3
x = 1.0
x
1.0
```

Variables don't need to be numbers

They can be any Python "thing"

```
x = "hello"
x
'hello'
type(x)
str
x = 1
type(x)
int
x = "hello "
y = "world"
x + y
'hello world'
```

```
x = True
y = False
x + y
```

```
1
```

Advanced topic: assignment with mutation

You can simultaneously assign and update a variable at the same time. Don't worry about this if it's confusing

```
x = 1
x = x + 2
x
```

```
3
```

```
x = 1
x += 2
x
```

```
3
```

```
x = "hello"
x += x
x
```

```
'hellohello'
```

```
x = 5
x -= 2
x
```

```
3
```

```
x = 2
x *= 3
x
```

```
6
```

```
x = 3
x /= 2
x
```

```
1.5
```

```
x = 3
x //= 2
x
```

```
1
x **= 2
x
1
```

Advanced variable naming

```
abc = 1
abc

1

xy = 2
x

1

xy
2

xy = 5
y

False

xy = 10
yz

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[112], line 2
      1 xy = 10
----> 2 yz

NameError: name 'yz' is not defined

variable_name = 1
variable_name

1

_var = 'hi'
_var

'hi'
```

```
0var = 1
```

```
Cell In[115], line 1
```

```
0var = 1
```

```
^
```

```
SyntaxError: invalid decimal literal
```

```
var0 = 1
```

```
var0
```

```
1
```

```
v0r = 1
```

```
v0r
```

```
1
```

```
v"r = 1
```

```
Cell In[118], line 1
```

```
v"r = 1
```

```
^
```

```
SyntaxError: unterminated string literal (detected at line 1)
```

```
v+r = 1
```

```
Cell In[119], line 1
```

```
v+r = 1
```

```
^
```

```
SyntaxError: cannot assign to expression here. Maybe you meant '=='  
instead of '='?
```

```
True = 1
```

```
Cell In[120], line 1
```

```
True = 1
```

```
^
```

```
SyntaxError: cannot assign to True
```

```
true = False
```

```
true
```

```
False
```

```
x = 3
```

```
y = 5
```

```
x = True
```

```
z = False
```

```
xy
```

```
10
```

```
2 ** 5
```

```
32
```

```
2 * 5
```

```
10
```

```
=
```

```
Cell In[125], line 1
```

```
=
```

```
^
```

```
SyntaxError: invalid syntax
```

```
x = = y
```

```
Cell In[126], line 1
```

```
x = = y
```

```
^
```

```
SyntaxError: invalid syntax
```

```
True == True
```

```
True
```

```
x = 1
```

```
x
```

```
1
```

```
1=a
```

```
Cell In[129], line 1
```

```
1=a
```

```
^
```

```
SyntaxError: cannot assign to literal here. Maybe you meant '=='  
instead of '='?
```

Printing

You may have noticed we need to put the variable at the end for Jupyter to show us what it contains.

We can get around this by printing.

```
print("Hello")
```

Hello

```
print("Hello")
```

x

Hello

1

```
x = 1
```

```
print(x)
```

```
x = 2
```

```
print(x)
```

x

1

2

2

```
print(1 + 1)
```

2

```
print(True)
```

True

```
x = 3
```

```
print(x)
```

```
y = 5
```

```
print(y)
```

```
x = True
```

```
print(x)
```

```
z = False
```

```
print(z)
```

xy

3

5

True

False

10

```
print(x)
```

True

```
print(z)
```

```
False
print("z")
z
print(True)
True
x = "z"
print(x)
z
letter_z = "z"
print(letter_z)
z
```

Comments

In Python/Jupyter, we can have text that doesn't run as code. These are called *comments* and are usually used to help other people understand your code.

```
# This is a comment. Nothing happens
x

'z'

x = 1 # You can put comments after normal code. It won't run
x

1

x#z = 1

1
```

Motivation for conditionals

Many times, we want the behavior of a program to depend on the value of a "thing". In other words, the program can change its behavior depending on a variable or user input.

As a simple example, let's say we want to assign "A" to the variable `letter_grade` if the `numeric_grade` is greater than 90. How can we do this?

```
numeric_grade = 91
letter_grade = "A"
letter_grade

'A'

numeric_grade = 91
letter_grade = "B"
letter_grade

'B'
```

Conditionals

As we saw above, we need to manually assign the `letter_grade`. We currently don't know how to change the behavior of the program based on the `numeric_grade`!

We can do this using *conditionals*. I'll first show some examples and talk about the syntax.

```
numeric_grade = 89

if numeric_grade > 90:
    letter_grade = "A"
else:
    letter_grade = "B"

letter_grade

'B'

numeric_grade = 91

if numeric_grade > 90:
    letter_grade = "A"
else:
    letter_grade = "B"

letter_grade

'A'
```

Note: it's quite annoying to copy the code! We'll see how to fix this later

Basic syntax for conditionals

The general syntax for conditionals is

```
if <expression that evaluates to a boolean>:  
    <expression>  
else:  
    <expression>
```

We will call <expression that evaluates to a boolean> a <condition>.

```
True == True
```

```
True
```

```
if True:  
    print(1)  
else:  
    print(2)
```

```
1
```

```
if False:  
    print(1)  
else:  
    print(2)
```

```
2
```

```
if 1 == 1:  
    print(1)  
else:  
    print(2)
```

```
1
```

```
if 1 == 2:  
    print(1)  
else:  
    print(2)
```

```
2
```

```
x = 1  
y = 1
```

```
if x == y:  
    print(1)  
else:  
    print(2)
```

```
1
```

```
x = 1  
y = 1
```

```
if x == y:
    print(1)
else:
    print(2)
```

1

More about conditionals

The expressions within the *body* of the `if` or `else` can be complex, beyond a single statement

```
x = 1

if 20 > 10:
    x = 2
    x += 3
    x -= 1
else:
    x = 5

x

4

x = 3
y = 2

if x > y:
    z = x + y
else:
    z = 1

z

5
```

Important note: the other branch is NOT executed

Only one branch of the `if` or `else` is executed. If a branch is not taken and a variable is not defined, you will have an error

b

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[158], line 1
----> 1 b

NameError: name 'b' is not defined

print(x)
print(y)

if x > y:
    a = 1
else:
    b = 1

print(a)
b

3
2
1

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[159], line 10
      7     b = 1
      9     print(a)
--> 10 b

NameError: name 'b' is not defined

```

Complex conditionals

With the `letter_grade` example, we only assigned grades of `A` and `B`. That's not very satisfying! Let's do more.

```

numeric_grade = 75

if numeric_grade > 90:
    letter_grade = 'A'
elif numeric_grade > 80:
    letter_grade = 'B'
elif numeric_grade > 70:
    letter_grade = 'C'

```

```

elif numeric_grade > 60:
    letter_grade = 'D'
else:
    letter_grade = 'F'

letter_grade
'C'

# General syntax for if / elif / else
if <condition>:
    <expression>
elif <condition>:
    <expression>
...
elif <condition>:
    <expression>
else:
    <expression>

Cell In[161], line 2
    if <condition>:
        ^
SyntaxError: invalid syntax

```

IMPORTANT: The ORDER of the if/elif matters!

```

numeric_grade = 75

if numeric_grade > 60:
    letter_grade = 'D'
elif numeric_grade > 90:
    letter_grade = 'A'
elif numeric_grade > 80:
    letter_grade = 'B'
elif numeric_grade > 70:
    letter_grade = 'C'
else:
    letter_grade = 'F'

letter_grade
'D'

```

IMPORTANT: You must have a tab for the body of the if/elif/else

```
if x > y:  
    print('hi')  
else:  
    print('bye')
```

```
Cell In[163], line 2  
    print('hi')  
    ^
```

IndentationError: expected an indented block after 'if' statement on line 1

```
print(x)  
print(y)
```

```
3  
2
```

```
if x > y:  
    print('hi')  
else:  
    print('bye')
```

```
hi
```

Note: you don't need the else

```
x = 1  
y = 2  
  
z = 3  
if x > y:  
    z = 4
```

```
z
```

```
3
```

```
x = 2  
y = 1
```

```
z = 3  
if x > y:  
    z = 4
```

z

4

Line 1

Line 2

```
if x > y:  
    print(1)  
elif y < x:  
    print(2)
```

1

x = 4

y = x * 2